Global Partitions for Boosting Time Series Classification

Matthew Yang

October 17, 2026

Abstract

Gradient boosted decision trees are the usual classifier to put on top of interval-based time series features, but inducing each tree means searching for splits over the whole training set. We replace the tree with a *global partition*: b threshold predicates, placed between pairs of hard, differently labelled examples, that index a table of 2^b leaf values. The resulting booster, HashBoost, does no exhaustive split search and trains from mini-batches, so its memory use does not depend on the size of the training set. We implement it on the GPU in PyTorch and evaluate it after a QUANT transform. On all 26 MONSTER datasets, with each model's configuration chosen by the same procedure on validation data and frozen before testing, HashBoost is level with tuned XGBoost on MONSTER's five test folds (error ratio 1.02, Wilcoxon $p = 0.98$) and trains 8.2× faster in total on the same GPU. Both boosters rank significantly ahead of QUANT's own ExtraTrees classifier there. On the 112 UCR datasets, where training sets are small, the order reverses: ExtraTrees is best, and HashBoost at its defaults is level with default XGBoost. With fewer bits per round and more rounds, HashBoost is significantly ahead of XGBoost. HashBoost's limit is the number of rounds it can afford, not memory: every batch updates every existing round, so cost is quadratic in rounds, and freezing old rounds makes it linear.

Contents

1 Introduction **2**

2 Background and Related Work **2**

3 Method **3**
3.1 A round is a global partition 3
3.2 Choosing the predicates 4
3.3 Fitting the leaves 4
3.4 Implementation 5

4 Experimental Setup **5**

5 Results **6**
5.1 MONSTER (RQ1) 6
5.2 UCR (RQ3) 8
5.3 Cost and scaling (RQ2) 9
5.4 Random partitions, and when to inform them 10

6 Discussion **11**

7 Conclusion **12**

1 Introduction

Gradient boosted models are built from a sequence of decision trees, where each tree is fitted to the residual errors of the ensemble constructed so far rather than to the original targets [1]. This stage-wise additive formulation, together with highly engineered implementations such as XGBoost [2], LightGBM [3] and CatBoost [4], has made boosting the default choice for tabular prediction tasks, where it remains competitive with, and frequently superior to, deep learning [5, 6].

Tree ensembles are also the usual classifier on top of interval-based time series features. QUANT [7] passes its quantile features to an ensemble of extremely randomised trees, and the features produced by PULSAR [8] and its predecessors are used the same way. At the scale of the MONSTER benchmark [9], with up to 5.9 million series per dataset, the classifier becomes the expensive part. Inducing a tree requires evaluating candidate split points at every node, so each boosting iteration costs time proportional to both the number of instances and the number of features. Histogram binning, approximate split finding, gradient-based sampling and feature bundling [2, 3, 10] all reduce the constant factor. They do not remove the need to revisit the whole training set in every round, and in practice to hold it in memory.

This paper asks whether the tree is needed at all. We replace each tree with a *global partition* of the feature space: b axis-aligned threshold predicates, applied to every example at once. The resulting b bits index a table of 2^b leaf values. A predicate takes a random feature and puts its threshold at the midpoint of two hard, differently labelled examples. Each leaf value is a Newton step computed from running sums of the gradient and hessian over the examples that land in it. Because the partition does not depend on which examples are in the current batch, those running sums can be updated from any later mini-batch, and the model never needs the training set in one place. We ask three questions:

RQ1 Can a booster of global partitions match a boosted tree ensemble's accuracy at scale, given the same tuning effort?

RQ2 What does it cost to train, and how does that cost grow with the training set and the number of rounds?

RQ3 How does it fare on small training sets, against the classifier QUANT was designed with?

Contributions.

1. **HashBoost**, a gradient booster whose weak learner is a global partition, and a GPU implementation that agrees with an independent CPU reference to 4×10^{-7} (Section 3).
2. **Round freezing**, which with a per-row cache turns HashBoost's quadratic cost in rounds into a linear one (Section 3.3).
3. **An evaluation on all 26 MONSTER datasets**, trained on whole pools of up to 4.65 million rows and scored on MONSTER's five test folds, against XGBoost tuned by the same procedure and against QUANT's ExtraTrees (Section 5.1).
4. **An evaluation on the 112 UCR datasets** against the same two baselines, including how the bit width and round budget should change for small training sets (Section 5.2).

2 Background and Related Work

Boosting. Boosting originates in the observation that a set of weak learners can be combined into an arbitrarily strong one, formalised by AdaBoost [11] and later generalised to arbitrary differentiable losses as gradient boosting [1]. Decision trees are the conventional weak learner,

but nothing in the gradient boosting framework requires them. Modern implementations fit each tree to a second-order expansion of the loss, so that a leaf's value is a Newton step $-\sum g / \sum h$ over the examples it contains [2]. HashBoost keeps this leaf rule and changes how examples are assigned to leaves.

Oblivious trees and decision tables. CatBoost [4] grows *oblivious* trees, in which every node at a given depth uses the same split. A depth-d oblivious tree is therefore d predicates whose outcomes index 2^d leaves. This is the same structure as one HashBoost round, and neural oblivious ensembles [12] use it too. CatBoost chooses each level's split greedily by gain over the whole training set, and fixes the leaves once the tree is built. Boosted decision tables [13] are the closest relative. They choose a table's tests greedily by squared-error gain over the whole training set, then *backfit* them: each test is removed and re-chosen given the others. A table's leaf values are set once, as the mean target in each partition, and earlier tables are never revisited. HashBoost does the opposite on both counts. Its tests are not searched for over the training set, and the leaves of every earlier round are re-estimated by each later batch. That is closer to backfitting additive models [14], applied online, than to backfitting the splits within one table.

Randomised partitions. Random forests [15] and extremely randomised trees [16] showed that much of the split search in a tree can be replaced by randomness without losing accuracy, provided the ensemble is large. Extremely randomised trees draw both the feature and the threshold at random, and QUANT uses them as its classifier [7]. A random global partition can also be read as a locality-sensitive hash [17]: examples that share a code agree on b random comparisons. HashBoost's thresholds are random but not uninformed. They sit between two hard examples of different classes, so each bit tends to separate examples the current model confuses.

Time series classification. Random-feature methods have shown that the expensive, data-dependent search of tree induction can often be replaced by cheap, data-independent random transformations. ROCKET [18] applies a large bank of random convolutional kernels followed by a linear classifier, matching far more expensive ensembles such as HIVE-COTE [19] at a small fraction of the cost, and MiniRocket [20] and Hydra [21] refine it. A parallel line of work extracts features from intervals of the series: Time Series Forest [22], CIF [23], DrCIF [19], rSTSF [24], QUANT [7] and PULSAR [8]. All of these end in some form of tree. We use QUANT as a fixed feature transform and replace only the classifier.

3 Method

3.1 A round is a global partition

Let $x \in \mathbb{R}^p$ be a feature vector and k the number of classes. A round r is defined by b pairs $(j_{r,i}, t_{r,i})$ of a feature index and a threshold, for $i = 0, \ldots, b-1$. It maps x to a b-bit code

$$c_r(x) = \sum_{i=0}^{b-1} 2^i \mathbb{1}\big[x_{j_{r,i}} \le t_{r,i}\big] \in \{0, \ldots, 2^b - 1\}, \tag{1}$$

and has a table $L_r \in \mathbb{R}^{2^b \times k}$ of leaf values. The model after R rounds is

$$F(x) = \sum_{r=1}^{R} L_r[c_r(x)] \in \mathbb{R}^k, \tag{2}$$

and predicts $\mathrm{softmax}(F(x))$. A round is an oblivious tree of depth b whose splits are fixed when it is created. The default is $b = 8$, so a round has 256 buckets.

Algorithm 1 One HashBoost step on a mini-batch (X, y) of n examples, adding H rounds.

for $h = 1, \ldots, H$ **do**
  $F \leftarrow \sum_{r<R} L_r[c_r(X)]$ ▷ (n, k) logits
  $p \leftarrow \mathrm{softmax}(F)$; $g \leftarrow p - \mathrm{onehot}(y)$; $h \leftarrow p(1-p) + 10^{-3}$
  choose predicates of round R from hard pairs in (X, y, p)
  for $r = 0, \ldots, R$ **do** ▷ all rounds, including the new one
    $G_r[c_r(X)] \mathrel{+}= -g$; $H_r[c_r(X)] \mathrel{+}= h$ ▷ scatter-add
    $L_r \leftarrow \eta\, G_r/(H_r + \epsilon)$
  end for
  $R \leftarrow R + 1$
end for

3.2 Choosing the predicates

Given the current batch, its probabilities and its labels, a new round's predicates are chosen as follows:

1. Order the batch by cross entropy, hardest first.
2. Walk that order and greedily form b pairs of examples with different labels.
3. For each pair (u, v), draw a feature j uniformly at random and set $t = (u_j + v_j)/2$.

Each bit therefore separates two examples the model currently gets wrong, and the only search is the sort in the first step. A *gain-selected* variant draws K such hard-pair candidates per bit and fills the bits one at a time, as CatBoost grows an oblivious tree: each bit takes the candidate with the highest second-order gain, $\sum G^2/(H + \epsilon)$, within the cells the earlier bits have made. It searches only K random candidates per bit, on one batch.

3.3 Fitting the leaves

We use the softmax cross-entropy gradient $g = p - \mathrm{onehot}(y)$ and a floored diagonal hessian $h = p(1-p) + 10^{-3}$. Every round keeps running sums $G_r, H_r \in \mathbb{R}^{2^b \times k}$ of $-g$ and h over the examples that landed in each bucket, and its leaf values are

$$L_r[s] = \eta \, \frac{G_r[s]}{H_r[s] + \epsilon}, \tag{3}$$

with learning rate $\eta = 0.1$. By default $\epsilon = 10^{-6}$; a larger ϵ is an L2 penalty that shrinks thinly populated leaves.

The key departure from tree boosting is *when* these sums are accumulated. A tree's leaves are fitted once, when the tree is built, and never change. In HashBoost, **every mini-batch is accumulated into every existing round**, with gradients evaluated at the current model (Algorithm 1). A round created early keeps absorbing evidence from every batch that follows, so a round is not tied to the batch that created it, and the model can learn from a stream. It is also where the cost comes from: a batch costs $O(R)$ once there are R rounds, so T batches with one new round each cost $O(T^2)$.

Rounds per batch. Adding $H > 1$ rounds per batch raises capacity without more passes over the data. Each of the H steps re-accumulates the batch into the existing rounds.

Round freezing. With a window of W rounds, only the newest W keep accumulating. Older rounds are frozen, and a frozen round's contribution to a given example never changes again. Given stable row ids, each row's frozen contribution is summed once, cached in an (N, k) table, and never recomputed. A batch then costs $O(W + f)$, where f is the number of rounds frozen since its rows were last seen, so total cost becomes linear in rounds.

3.4 Implementation

HashBoost runs entirely on the GPU in PyTorch. A batch's codes are a `uint8` tensor of shape (R, n), built one bit at a time, so the largest intermediate is a single gather. G and H share one $(R, 2^b, 2k)$ tensor, so one `scatter_add_` updates both. Work is chunked over rounds to a fixed memory budget, so peak memory does not grow with R beyond the tables themselves. The original numba CPU implementation is kept as a reference. Given the same predicates, the two agree to 4×10^{-7} in logits with identical predictions, and a test suite keeps the defaults equal to it. The GPU version trains Pedestrian's default 800 rounds in about 7.5 s, against 101.5 s for the reference.

4 Experimental Setup

MONSTER. We use all 26 classification datasets in MONSTER [9] (Table 1), with QUANT [7] features (depth 6, div = 4, our PyTorch port). MONSTER supplies five fixed test folds. On fold k the *pool* is every series outside test fold k. A fixed shuffle of the pool (seed 42) gives a validation slice and a tune slice of 4,096 rows each, or a fifth of the pool each when it has fewer than 73,728 rows, and every model trains on all the rest. The tune slice is used for every choice made after training, such as early stopping. Validation error is used only to choose configurations.

Configurations. HashBoost and XGBoost were tuned the same way, on fold 0's validation error alone. We screened one change at a time on 10 of the 26 datasets (CrowdSourced, DREAMERA, FordChallenge, InsectSound, LenDB, PAMAP2, STEW, Traffic, UCIActivity and WISDM), fixed two configurations per model, and ran both on all 26. For each dataset and model the configuration with the lower fold-0 tune error was frozen. Each frozen configuration was then trained once on each of the five folds and scored once on that fold's test series, at the round its tune error chose. No test series was seen before this.

Both HashBoost configurations train for up to 5,000 batches of 4,096 rows, evaluate every 50 batches, stop once their best tune evaluation is 1,000 batches old, and freeze rounds older than 128 batches:

- `G`: 6-bit gain-selected rounds ($K = 64$ candidates per bit), $\epsilon = 100$, 8 rounds per batch. Picked on 18 datasets.
- `L`: 8-bit hard-pair rounds, $\epsilon = 1$, 16 rounds per batch. Picked on the largest pools (LenDB, both S2Agri, Traffic, Pedestrian), WhaleSounds and the EEG sets DREAMERA and DREAMERV.

Over five model seeds on fold 0, the pick was the same every time on 25 of the 26 datasets. XGBoost's screen raised its cap to 5,000 rounds with 100 rounds of patience, then varied depth, learning rate, subsampling, L2 regularisation and the number of bins. Its configurations are `d4` (depth 4, learning rate 0.1; picked on 17 datasets) and `d8` (depth 8, learning rate 0.05, 64 bins; picked on 9). The third model, ExtraTrees with QUANT's own settings (200 trees, entropy criterion, a tenth of the features per split), chooses nothing after training, so it trains on the same rows and needs no tune slice.

Table 1: The 26 MONSTER datasets. "Pool" is the number of series outside test fold 0, and "train" the rows trained on after the validation and tune slices are taken from it. Classes are those present in the labels.

dataset	series	pool	train	classes	QUANT features
AudioMNIST-DS	30,000	24,000	15,808	10	35,349
CornellWhaleChallenge	30,000	24,000	15,808	2	35,349
CrowdSourced	12,289	9,456	5,674	2	31,542
DREAMERA	170,246	140,638	132,446	2	31,542
DREAMERV	170,246	140,638	132,446	2	31,542
FordChallenge	36,257	29,003	20,811	2	14,130
FruitFlies	34,518	27,614	19,422	3	44,031
InsectSound	50,000	40,000	32,768	10	5,470
LakeIce	129,280	103,424	95,232	3	1,652
LenDB	1,244,942	983,483	975,291	2	14,940
MosquitoSound	279,566	223,652	215,460	6	33,178
Opportunity	17,386	12,184	7,312	5	117,746
PAMAP2	38,856	28,961	20,769	12	54,184
Pedestrian	189,621	151,696	143,504	82	212
S2Agri-10pc-17	5,850,881	4,661,858	4,653,666	17	2,120
S2Agri-10pc-34	5,850,881	4,661,858	4,653,666	29	2,120
STEW	28,512	21,384	13,192	2	31,542
Skoda	14,117	11,290	6,774	11	62,520
TimeSen2Crop	1,135,511	839,971	831,779	15	30,096
Tiselac	99,687	79,907	71,715	9	2,040
Traffic	1,460,968	1,168,774	1,160,582	7	212
UCIActivity	10,299	7,898	4,740	6	10,440
USCActivity	56,228	43,060	34,868	12	6,252
WISDM	17,166	11,989	7,195	6	3,126
WISDM2	149,034	98,094	89,902	6	3,126
WhaleSounds	105,163	84,130	75,938	8	22,105

UCR. We also use the 112 univariate UCR datasets [25] with equal lengths and no missing values that the bake-off reports on [26], less Fungi, which has one training series per class. Each uses the archive's default train/test split. The training sets are small: 16 to 8,926 series (median 190), and only 11 have 1,000 or more. UCR has no validation split, so every model runs at fixed settings, nothing is chosen after training, and we report test error. The baselines are ExtraTrees with QUANT's settings and XGBoost at its library defaults (100 rounds, learning rate 0.3, depth 6). HashBoost's default budget is 800 rounds of 8 bits, cycling batches of at most 4,096 rows.

Hardware and noise. The MONSTER boosting runs each ran alone on one NVIDIA L40S (46 GB) on Monash's M3 cluster, and ExtraTrees on CPU nodes with 36 cores. UCR and the streaming and freezing runs in Section 5.3 used one RTX 4060 (8 GB). Seeding does not make HashBoost reproducible: float32 scatter-adds on the GPU accumulate in a nondeterministic order, and the perturbation grows through the ranking that chooses predicates. On Pedestrian the run-to-run standard deviation of validation error is about 0.003, so gaps under about 0.006 there are not treated as results.

5 Results

5.1 MONSTER (RQ1)

Table 2 gives each model's test error on the 26 MONSTER datasets, averaged over the five test folds, and Table 3 summarises the pairwise comparisons. Every restored model reproduced the

Table 2: MONSTER: test error, mean of the five test folds, one run per fold. HashBoost and XGBoost use the configuration each picked on fold 0. Times are training seconds summed over the five folds, QUANT included: HashBoost and XGBoost on one L40S, ExtraTrees on 36 CPU cores. The lowest error in each row is bold. †One fold rerun with a fixed intercept (footnote 1).

	HashBoost		XGBoost		ExtraTrees	time (s)		
dataset	config	error	config	error	error	HB	XGB	ET
AudioMNIST-DS	G	**0.0816**	d4	0.0884	0.1270	503	885	429
CornellWhaleChallenge	G	0.1250	d8	**0.1233**	0.1262	290	424	412
CrowdSourced	G	0.3501	d4	0.3622	**0.3396**	192	95	119
DREAMERA	L	0.5161	d8	0.5312	**0.4867**	438	1,775	4,253
DREAMERV	L	**0.4036**	d8	0.4038	0.4179	434	2,264	4,235
FordChallenge	G	0.0823	d8	0.0731	**0.0715**	196	152	224
FruitFlies	G	**0.0458**	d4	0.0462	0.0619	383	384	670
InsectSound	G	**0.1872**	d4	0.1891	0.2268	520	357	148
LakeIce	G	**0.0018**	d4	0.0019	0.0024	195	38	116
LenDB	L	0.0538	d4	**0.0522**	0.0551	728	1,364	25,843
MosquitoSound	G	0.1052	d4	**0.1008**	0.1596	1,855	5,054	7,366
Opportunity	G	0.1322	d4	**0.1305**	0.1394	399	852	761
PAMAP2	G	**0.2143**	d4	0.2246	0.2288	277	1,025	1,013
Pedestrian	L	**0.1746**	d8	0.1772	0.2135	2,588	792	61
S2Agri-10pc-17	L	0.2829	d8	0.2525	**0.2521**	1,463	30,068	18,291
S2Agri-10pc-34	L	0.2828	d8	**0.2530**	0.2638	2,013	61,933	16,151
STEW	G	**0.1937**	d4	0.1993	0.2371	325	310	382
Skoda	G	0.0567	d4	0.0610	**0.0565**	354	784	293
TimeSen2Crop	G	0.1412	d4	**0.0849**	0.1808	1,926	35,655	32,910
Tiselac	G	**0.2139**	d4	0.2140	0.2172	396	268	115
Traffic	L	0.2841	d8	0.2939	**0.2488**	846	407	375
UCIActivity	G	**0.0521**	d4	0.0675	0.0855	169	46	21
USCActivity	G	0.2428	d4	**0.2396**	0.2751	497	305	188
WISDM	G	0.1152	d4	**0.1131**†	0.1298	161	22	17
WISDM2	G	0.2556	d4	**0.2541**	0.2614	458	206	297
WhaleSounds	L	0.3128	d8	**0.2802**	0.2936	612	3,633	1,856
total						18,214	149,099	116,546

validation error it had recorded at its tune-chosen round, which checks that test error was scored at that point.

HashBoost and tuned XGBoost are level. HashBoost's error is 1.016 times XGBoost's in geometric mean. It is lower on 14 datasets and higher on 12, and lower on 61 of the 130 dataset-folds. The Wilcoxon test finds no difference ($p = 0.98$)[1]. Where one model clearly leads, it is usually XGBoost: on TimeSen2Crop (HashBoost's error is 1.66 times XGBoost's), FordChallenge, WhaleSounds, both S2Agri subsets and MosquitoSound. HashBoost clearly leads only on AudioMNIST-DS. The 10 tuning datasets flatter HashBoost, which was developed on them over many more rounds than XGBoost: its ratio there is 0.973, against 1.045 on the other 16.

[1]One XGBoost run failed. WISDM's test fold 3 holds every series of two of its six classes, so that fold's pool lacks them, and XGBoost 3.4's estimate of each class's starting score from the training labels came out as +3.4 and +4.8 for the absent classes and −8.1 for the two commonest. Training never recovered (test error 0.683). We report the rerun with the intercept fixed at 0.5, XGBoost's default before version 2.0, which reaches 0.178. Counting the failed run instead, the ratio is 0.992 ($p = 0.71$).

Table 3: MONSTER: pairwise comparisons over the 26 datasets. The ratio is the geometric mean of the first model's error over the second's. W/T/L counts the datasets where the first has the lower, equal and higher five-fold mean. "Clear" counts those where a 95% paired-t interval over the five per-fold differences excludes zero, either way. p is a two-sided Wilcoxon signed-rank test on the five-fold means, Holm-adjusted over the three comparisons.

comparison	ratio	W/T/L	clear (W/L)	p
HashBoost vs XGBoost	1.016	14/0/12	1/6	0.98
HashBoost vs ExtraTrees	0.904	18/0/8	9/5	0.11
XGBoost vs ExtraTrees	0.890	20/0/6	13/3	0.031

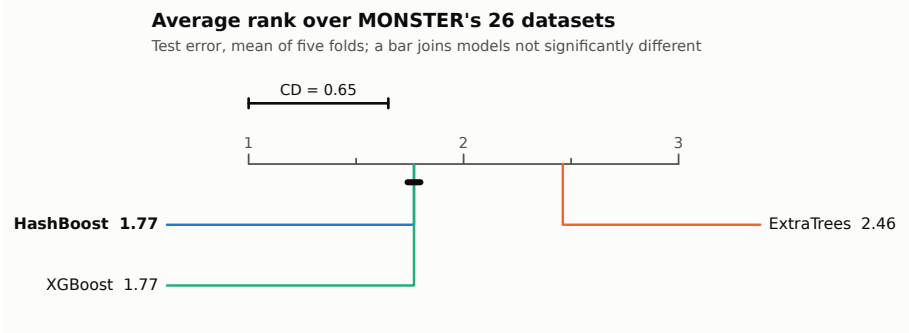


Figure 1: Critical difference diagram over MONSTER's 26 datasets [27]: test error, mean of the five folds. A Friedman test rejects equal ranks ($p = 0.016$). The bar joins models within the Nemenyi critical difference (0.65) of each other at $\alpha = 0.05$.

Both boosters beat ExtraTrees. The mean ranks are 1.77 for both boosters and 2.46 for ExtraTrees (Figure 1). ExtraTrees is 0.69 behind, just beyond the Nemenyi critical difference of 0.65. Its largest losses are on big acoustic and sensor pools: MosquitoSound (1.52 times HashBoost's error), AudioMNIST-DS, FruitFlies, Pedestrian, InsectSound and TimeSen2Crop. It clearly leads both boosters only on Traffic. On UCR the order is reversed (Section 5.2).

Test error is about 3.3 times validation error, for all three models. The validation and tune slices are random draws from the pool, while the test folds differ from it in distribution. On DREAMERA, XGBoost's validation error is half HashBoost's (0.071 against 0.135), yet both are near 0.5 on test. Every model is tuned on the same optimistic slices, so the comparison stays fair, but on MONSTER validation gaps do not carry over to test gaps.

5.2 UCR (RQ3)

Table 4 summarises the three models on the UCR 112, one run each on the default splits. Our ExtraTrees reproduces the published QUANT results [26]: mean error 0.1475 against 0.1456, with a correlation of 0.996 across datasets.

At its defaults, HashBoost is level with XGBoost and behind ExtraTrees. ExtraTrees is significantly better than both boosters (Figure 2), and HashBoost has the lower error on only 8 datasets against it. HashBoost and XGBoost are not significantly different (mean ranks 2.36 and 2.39). The record against XGBoost depends on training-set size. On the 58 datasets with at most 200 training series HashBoost wins 39 and loses 17, where XGBoost's defaults do badly. Above 200 it wins 19 and loses 34, and on the 11 with at least 1,000 it trails both baselines.

Table 4: UCR 112: test error on the default splits, one run per dataset. Mean rank is among the three models we ran. W/T/L is HashBoost's record against each row. p is a two-sided Wilcoxon signed-rank test against HashBoost, Holm-adjusted over the four comparisons. The published rows are the bake-off's resample 0, which is the default split.

model	mean error	median	mean rank	W/T/L	p	total fit
HashBoost	0.1941	0.1705	2.36	–	–	183 s
ExtraTrees	**0.1475**	0.1205	**1.25**	8/8/96	2.6×10^{-14}	70 s
XGBoost	0.2101	0.1912	2.39	58/3/51	0.082	413 s
QUANT (published)	0.1456	0.1177		13/5/94	4.9×10^{-14}	
HC2 (published)	0.1236	0.0753		8/7/97	3.8×10^{-15}	

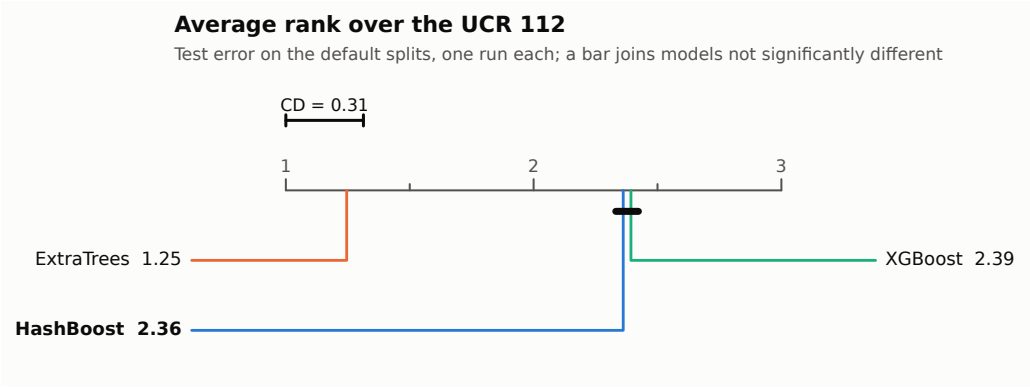


Figure 2: Critical difference diagram over the UCR 112 [27]. A Friedman test rejects equal ranks ($p = 3.6 \times 10^{-22}$). A bar joins models within the Nemenyi critical difference (0.31) of each other at $\alpha = 0.05$.

Fewer bits and more rounds. The default of 8 bits was set on MONSTER, where each of a round's 256 buckets sees thousands of rows. Most UCR training sets cannot fill them. Table 5 varies the width and the budget, with the default rerun in the same sweep as the control. Every other cell beats the control (Holm-adjusted Wilcoxon $p \leq 5.2 \times 10^{-7}$): fewer bits help at both budgets, and more rounds help at every width. Because the grid is scored on test sets, we choose a cell by two-fold cross-validation over datasets, picking it on one half and scoring it on the other, both ways round. Both halves choose two bits and 3,200 rounds. Chosen that way, HashBoost is significantly ahead of default XGBoost (lower error on 82 datasets, higher on 29, $p = 4.5 \times 10^{-8}$) and still behind ExtraTrees (28 against 63, $p = 1.2 \times 10^{-5}$), though the gap in mean error falls from 0.048 to 0.017. The best width grows with the training set: at 3,200 rounds two bits is best below 1,000 training series and four bits above.

5.3 Cost and scaling (RQ2)

Training time on MONSTER. On the same GPU, HashBoost trained 8.2 times faster than XGBoost in total: 18,214 s against 149,099 s over the 130 dataset-folds, or 3.9 times without S2Agri, where XGBoost spent 92,001 s. On small pools XGBoost is faster, because HashBoost's cost per round is mostly fixed: UCIActivity took 169 s against 46 s. Figure 3 shows where the time goes on the six largest pools. On S2Agri-10pc-34, HashBoost reaches its tune-chosen point (validation error 0.0200) after 375 s and XGBoost reaches its point (0.0166) after 12,661 s. On TimeSen2Crop the times are 398 s and 5,256 s. There XGBoost's extra hours buy its accuracy lead, but HashBoost was also stopped by its budget: it reached 5,000 batches on 14 of the 15 S2Agri and Traffic folds with its best tune error in the last 1,000 batches, still improving. On Traffic and Pedestrian its curve is at or below XGBoost's throughout.

Table 5: UCR 112: HashBoost's mean test error by bits per round and round budget, one run per cell. The default (8 bits, 800 rounds) was rerun in the same sweep. The grid is scored on the test sets, so the best cell's own figure is optimistic; the text chooses a cell without reading the test sets it is scored on.

bits	800 rounds	3,200 rounds
8 (default)	0.1956 (control)	0.1885
6	0.1844	0.1808
4	0.1767	0.1712
2	0.1750	**0.1647**

Table 6: Round freezing on Pedestrian, 65,536 rows, 3 runs, one sweep. The window is in epochs of 16 batches.

rounds per batch	window	validation error	wall
1	none	0.2252 ± 0.0023	8.4 s
1	10 epochs	0.2276 ± 0.0037	3.6 s
1	2 epochs	0.2332 ± 0.0042	1.8 s
2	none	0.2133 ± 0.0025	25.6 s
2	10 epochs	$\mathbf{0.2135} \pm 0.0032$	**10.8 s**

Memory does not grow with the training set. We streamed LenDB's whole training pool (975,291 rows: 6.3 GB as raw series and 58 GB after QUANT) through both models on the RTX 4060, on the same validation rows as a 65,536-row run. HashBoost's peak GPU memory was the same at both sizes, 2,572 MB, because each batch is folded into the tables and freed. XGBoost's binned matrix is one byte per row and feature and grows linearly (Figure 4, right): 14.6 GB at the full pool, which it paged from disk every round at 116 s per round against 0.68 s at 65,536 rows. On the L40S in the MONSTER benchmark, tuned XGBoost held LenDB's matrix on the card at 17.1–17.8 GB, against 3.4–4.9 GB for HashBoost's `L`.

More data needs more rounds. With the default single round per batch, the whole-pool model was *worse* than the 65,536-row model at the same 1,200 rounds (validation error 0.0691 against 0.0532). It was underfitting, with a training error of 0.0608, while the small model had memorised its rows (0.0049). With 16 rounds per batch and freezing, `L` reaches 0.0428 ± 0.0018 over five seeds on the whole pool, ahead of both. HashBoost's scaling is limited by rounds, not memory, and the quadratic cost is what makes rounds expensive.

Freezing makes the cost linear. Per-batch cost grows with the number of rounds: on a streamed Traffic run it rose from 2 ms to 14.5 ms over 2,840 rounds, while a window of one or two epochs' rounds levels it off (Figure 4, left), and 1,600 rounds on Pedestrian took 37.5 s against 10.5 s for 800. Table 6 shows freezing on Pedestrian. A window of ten epochs' rounds matches 1,600 unfrozen rounds' accuracy in 2.4 times less time. The window matters: two epochs cost up to 0.027 on Pedestrian, but nothing measurable on InsectSound or on whole-pool Traffic and LenDB. Every configuration in Section 5.1 freezes rounds older than 128 batches.

5.4 Random partitions, and when to inform them

In development at 65,536 rows, nearly every change that made an individual round better informed made the ensemble worse. Choosing each round's predicates by gain raised the number of occupied buckets from 79 to 119 of 256 on Pedestrian, and left the error the same or worse. The same went for richer QUANT features, feature filtering and oblique predicates. The randomness

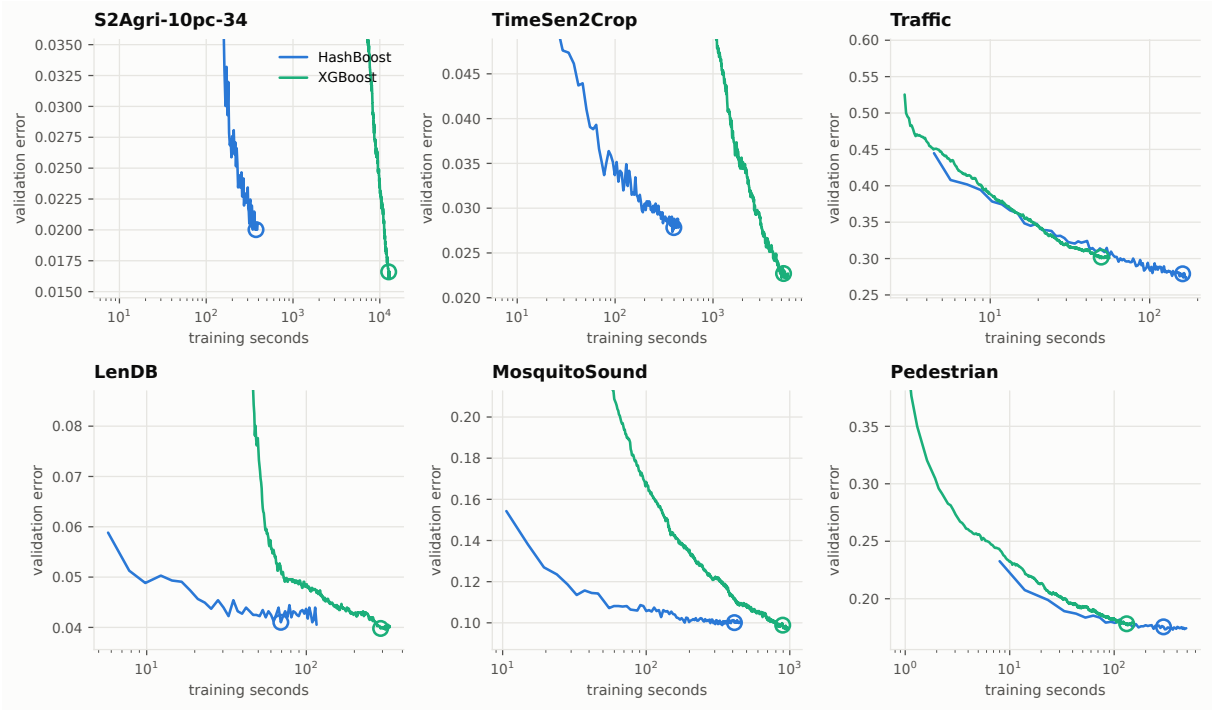


Figure 3: Validation error against training time on fold 0 for the six largest MONSTER pools, each booster at its frozen configuration, on one L40S, QUANT included. HashBoost's time is measured at each evaluation; XGBoost's is its build plus its mean time per round. Rings mark the tune-chosen point. The horizontal axis is logarithmic.

of the predicates appears to be what makes rounds diverse enough to add up, as in extremely randomised trees [16]. The exception came on whole MONSTER pools, where gain selection over 64 candidates, with 6-bit rounds and a heavy leaf penalty ($\epsilon = 100$), was the largest single gain in the screen: it cut CrowdSourced's validation error from 0.042 to 0.006 and FordChallenge's from 0.105 to 0.062. This became `G`. The penalty appears to be what lets gain-chosen rounds generalise, and where it does not (the largest pools and the EEG sets), the random `L` is picked instead.

6 Discussion

Can the tree be replaced? (RQ1, RQ3) At MONSTER's scale, yes. With both boosters tuned the same way and scored on the five test folds, HashBoost is level with XGBoost across all 26 datasets. On the 112 small UCR datasets it is level with default XGBoost at its own defaults, and ahead with fewer bits and more rounds. Neither booster is the best classifier for QUANT's features everywhere: ExtraTrees wins on UCR and loses on MONSTER. On small training sets an average of random trees wins, and on large ones a booster wins, whether its weak learner is a tree or a global partition.

What does it cost? (RQ2) HashBoost's memory use is independent of the training set, so it trains on whole pools of millions of rows on a single GPU, at an eighth of tuned XGBoost's total time on MONSTER. Its binding constraint is the number of rounds. Freezing removes the quadratic growth, but the round budget still limited it on the largest pools.

Limitations.

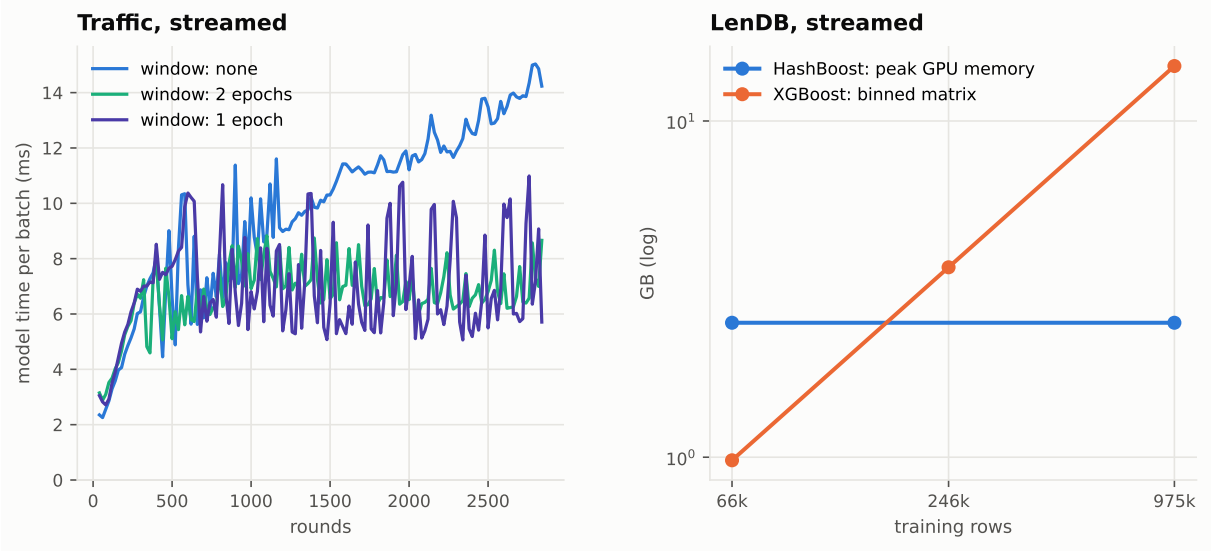


Figure 4: Left: HashBoost's model time per batch against rounds on a whole-pool Traffic stream (1.16 million rows, one round per batch, RTX 4060), unfrozen and with rounds older than one or two epochs frozen. Right: what each model needs available as LenDB's training pool grows. HashBoost's peak GPU memory is measured; XGBoost's binned matrix is measured on disk at 245,760 and 975,291 rows and derived from the same bytes per element at 65,536.

- The MONSTER benchmark is one run per test fold. Over five model seeds on fold 0, HashBoost's standard deviation was 0.0002 to 0.0024 on most datasets.
- Both models' configurations were chosen on 10 of the 26 MONSTER datasets, where HashBoost had also been developed for longer, and it does better there.
- HashBoost's budget of 5,000 batches stopped it while it was still improving on S2Agri and Traffic. A budget scaled with the pool might narrow its deficit on S2Agri; it was not tried, to keep the configurations frozen.
- On MONSTER, validation error underestimates test error about 3.3-fold, so configurations chosen on validation may not be the best on test, for either model.
- The UCR results are one run on the default split, where the bake-off averages 30 resamples, and its baselines are at fixed library settings. The bits-and-rounds choice never reads the test sets it is scored on, but it does read the other half's.
- The final comparison includes XGBoost and ExtraTrees only, not LightGBM or CatBoost. ExtraTrees ran on CPUs, so its time is not directly comparable.

Future work. A round budget that scales with the training set; a bit width chosen per dataset from its rows per class, which would unify the UCR and MONSTER settings; cheaper rounds on small datasets, where a round's cost is fixed overhead; and understanding the chaotic leaf dynamics that set HashBoost's run-to-run noise.

7 Conclusion

HashBoost replaces the decision tree in gradient boosting with a global partition: b threshold predicates, placed between pairs of hard examples, that index a table of Newton-step leaves refined by every later mini-batch. It never holds the training set. On all 26 MONSTER datasets,

scored on the five test folds, it is level with XGBoost tuned by the same procedure and trains 8.2 times faster on the same GPU, and both rank significantly ahead of QUANT's ExtraTrees. On the small UCR datasets, with fewer bits and more rounds, it is significantly ahead of XGBoost and behind ExtraTrees. What limits it is the number of rounds it can afford, which round freezing makes linear in cost.

References

[1] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, pp. 3146–3154, 2017.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31*, pp. 6638–6648, 2018.

[5] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems 35: Datasets and Benchmarks Track*, 2022.

[7] A. Dempster, D. F. Schmidt, and G. I. Webb, "QUANT: A minimalist interval method for time series classification," *Data Mining and Knowledge Discovery*, 2024.

[8] N. Cabello and L. Kulik, "Pulsar: Advancing interval-based time series classification to state-of-the-art performance," in *2025 IEEE International Conference on Data Mining (ICDM)*, pp. 1085–1094, 2025.

[9] A. Dempster, N. M. Foumani, C. W. Tan, L. Miller, A. Mishra, M. Salehi, C. Pelletier, D. F. Schmidt, and G. I. Webb, "Monster: Monash scalable time series evaluation repository," 2025.

[10] J. H. Friedman, "Stochastic gradient boosting," *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, 2002.

[11] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[12] S. Popov, S. Morozov, and A. Babenko, "Neural oblivious decision ensembles for deep learning on tabular data," in *International Conference on Learning Representations*, 2020.

[13] Y. Lou and M. Obukhov, "BDT: Gradient boosted decision tables for high accuracy and scoring efficiency," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1893–1901, 2017.

[14] A. Buja, T. Hastie, and R. Tibshirani, "Linear smoothers and additive models," *The Annals of Statistics*, vol. 17, no. 2, pp. 453–510, 1989.

[15] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[16] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[17] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pp. 604–613, 1998.

[18] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.

[19] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, "HIVE-COTE 2.0: A new meta ensemble for time series classification," *Machine Learning*, vol. 110, no. 11, pp. 3211–3243, 2021.

[20] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, (New York), pp. 248–257, ACM, 2021.

[21] A. Dempster, D. F. Schmidt, and G. I. Webb, "Hydra: Competing convolutional kernels for fast and accurate time series classification," *Data Mining and Knowledge Discovery*, vol. 37, pp. 1779–1805, 2023.

[22] H. Deng, G. Runger, E. Tuv, and M. Vladimir, "A time series forest for classification and feature extraction," *Information Sciences*, vol. 239, pp. 142–153, 2013.

[23] M. Middlehurst, J. Large, and A. Bagnall, "The canonical interval forest (CIF) classifier for time series classification," in *2020 IEEE International Conference on Big Data (Big Data)*, pp. 188–195, 2020.

[24] N. Cabello, E. Naghizade, J. Qi, and L. Kulik, "Fast, accurate and interpretable time series classification through randomization," 2021.

[25] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[26] M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake off redux: A review and experimental evaluation of recent time series classification algorithms," *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, 2024.

[27] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.